

D6 — Sprint Report

Each team submits one sprint report summarizing Sprint 0.

Accepted formats: Markdown pushed to the repository, PDF, or Word document.

If submitting to the repo: push to docs/sprint0/report.md (or report.pdf, report.docx).

Required sections:

Section	What to include
Team	Team name: Out of our minds member names and GitHub usernames: - Conner Aylward / cpaylward - Lauren Kim / laurenkim108 - Leana Gyalokay / TheOneWhoWished - Mustafa Aljassar / maljassar7 - Michael Wang/ Wung8 project name and repository link: outline https://github.com/CSCI-435-SE/outline
Project overview	Summary of AGENTS.md Outline is a collaborative knowledge base for teams, built with React and TypeScript in both frontend and backend. It uses a real time collaboration engine, and is designed for excellent performance and user experience. The backend is a Koa server with an RPC API and uses PostgreSQL and Redis. The application can be self-hosted or used as a cloud service. It needs these dependencies installed before running Use yarn for all dependency management. After updating dependency versions, install to update lockfiles Summary of ARCHITECTURE.md Outline has a backend and frontend codebase in this monorepo, both are written in TypeScript and share some code. Outline's frontend is a React application compiled with Vite. It uses MobX for state management and Styled Components for component styles. Unless global, state logic and styles are always co-located with React components together with their subcomponents to make the component tree easier to manage. The API server is driven by Koa, it uses Sequelize as the ORM and Redis with Bull for queues and async event management. Authorization logic is contained in cancan policies under the "policies" directory.

Feature backlog summary	How many issues were created: 28 Themes or categories: enhancement, feature, bug Top 3–5 features the team is most interested in pursuing during the semester: Document review/approval workflow, template gallery for documents, scanning for credentials Link to the issues. Issues · CSCI-435-SE/outline
Standards document summary	Key conventions adopted; any deviations from the project's existing guidelines and why: Unlike the original repo, we are okay with using AI-generated code. We also don't require a conversation in discussions to get to know one's process better; only a pull request is necessary. However, tests must be written for any new code, and if possible, manual tests (actually running it locally to make sure it works) are needed too. Link to standards.md:
Completed PRs	Table: PR title, issue link, author, reviewer(s), status (merged / open), brief description of the change
AI tool usage	Per team member: tools used, types of tasks, total sessions, link to ai-logs/sprint0/<username>/ folder. Notable observations about what worked or didn't. Conner Aylward Tools used: Claude Types of tasks: Enhancements and general understanding of the requirements Total sessions: 2 (Each sprint I will make a new session to keep the logs easily separate for review) ai-logs/sprint0/cpaylward Notable observations: Using claude code was far more helpful than the just claude premium by itself. I will probably stop using the browser model and just use claude code in the future. Lauren Kim: Tools used: Claude Code in WSL Types of tasks: Enhancements, implementation of changes, testing Total sessions: 2 ai-logs/sprint0/laurenkim108/2026-09-16_claude-code_enter-to-submit-comment.md ai-logs/sprint0/laurenkim108/2026-09-16_claude-code_enter-to-submit-comment.md Notable observations: What I found to work with Claude Code was that it was super detailed with its explanations as opposed to just giving an answer.

Leana Gyalokay:

Tools used: Claude in VS code

Types of tasks: Codebase understanding, solution planning, implementation, testing

Total sessions: 1

ai-logs/sprint0/TheOneWhoWished/2026-09-08_claude-code_issues-9-and-2.md

Notable observations: Claude is most effective when it is asked to consider multiple plans. It is still prone to hallucinations and needs to “second guess” itself before I can trust it.

Mustafa Aljassar:

Tools used: Claude

Types of tasks: Program running and server connection, error identification, testing, file location

Total sessions: 1

Notable observations: Good level of detail and tends to correct itself and spot its own mistakes rather than continue down an incorrect path.

Michael Wang:

Tools used: Claude Desktop

Types of tasks: codebase understanding, solution planning, implementation, testing

Total sessions: 2

Notable observations: Opus 5 was frontloading a lot of information, but was fine with answering specific questions about the codebase / how to implement changes. It did make some changes that weren't talked about beforehand and didn't fully explain but I think that's to be expected.

Release	Tag name and link to the Sprint 0 release on GitHub (see D7).
Risks and challenges	Communication and setting up deadlines have been a challenge. Sprint 1 may be slowed down by a lack of documentation and overall test suite
Sprint 1 ideas	Adding new features regarding visibility and organization such as activity feeds for collections, filtering, attaching reasons for archiving documents, and keeping the original folder structures visible in the archive.

Questions to answer:

What does the system do? Who are its users? What are its main features?

This is a software similar to Notion, but with live editing from multiple users similar to Google Docs. Many types of people may use this platform such as students, employees, or anyone who wants to organize thoughts with others. It is a text editor that allows users to make pages with embedded media and code organized into folders. The owner of a page can invite other users to edit the page, and they can work on the same page simultaneously.

What are the main components and how do they interact?

Outline is built from several connected parts that make collaboration smooth. The frontend, created with React, lets users write and organize documents. It communicates with the backend, which manages authentication, permissions, and data storage. The database (often PostgreSQL) keeps user and document information secure and consistent. A real-time engine uses WebSockets to sync edits between users instantly, while file storage and integrations link Outline to tools like Slack or Google Drive. Together, these components allow users to edit, share, and store content seamlessly in one workspace.

What are the major technologies, frameworks, and external services?

Important folders:

app/ — React web client, MobX for state management, Styled Components for styling.

Subfolders: actions, components, editor, hooks, menus, models (MobX observables), routes, scenes (full-page views), stores, types, utils.

server/ — Koa API server. Subfolders: routes/api and routes/auth, commands (multi-model business logic), models (Sequelize), policies (cancan-style authorization), presenters (DB model → JSON for the frontend), queues (Bull-based background jobs: processors react to events, tasks are ad-hoc jobs), migrations, emails, services (entry points like web, worker, websockets, collaboration, cron, admin).

shared/ — code used by both sides: the ProseMirror-based editor, i18n locale files, shared styles/colors, shared utils.

plugins/ — pluggable integrations, each self-contained: e.g. slack, github, gitlab, google, azure, notion, linear, zapier, webhooks, discord, figma, diagrams, oidc, passkeys, storage, plus analytics (googleanalytics, matomo, umami) and an enterprise plugin.

public/ — static assets served directly.

How is the code organized (directory structure, key packages/modules)?

app/ — React frontend (Vite, MobX, Styled Components):

- actions — reusable commands (navigate, open, create, publish)
- components — reusable React components across scenes
- editor — React components specific to the editor UI, plus editor extensions
- hooks — reusable React hooks
- menus — context menus reused across the app
- models — MobX observable state models (Document, Comment, Revision)

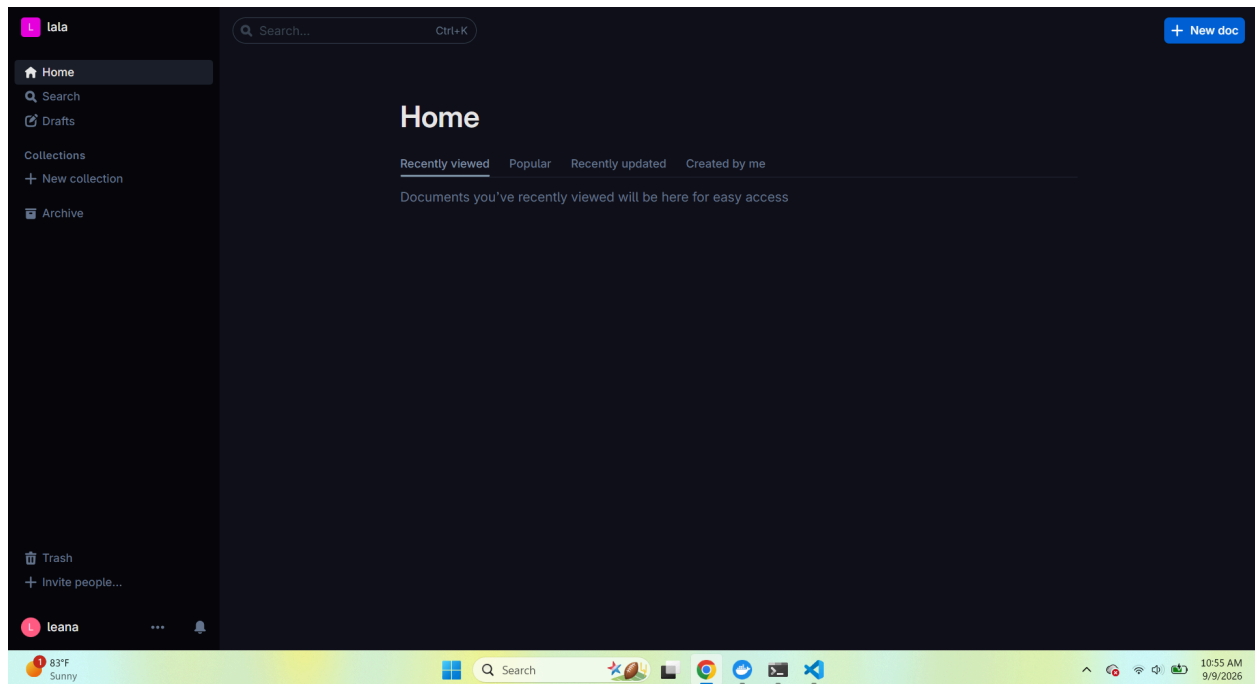
- routes — route definitions, async-loaded with suspense
- scenes — full-page views composed of components
- stores — collections of models plus their fetch/save logic
- server/ — Koa API (Sequelize + Postgres, Redis/Bull for queues):
- routes — API routes (routes/api/*) and auth routes (routes/auth/*)
- commands — actions spanning multiple models (e.g. documentUpdater, revisionCreator)
- models — Sequelize models
- policies — cancan-style authorization checks
- presenters — shape DB models into JSON API responses
- queues/processors — react to events on the event bus (e.g. RevisionsProcessor)
- migrations — Sequelize DB migrations
- shared/ — used by both frontend and backend:
- editor — the ProseMirror rich-text editor itself (nodes, marks, extensions, plugins)
- components — shared React components
- i18n — translations (auto-extracted, never hand-edited)
- styles — colors and global aesthetics

Other top-level: plugins/ (self-contained integrations — Slack, GitHub, Notion, SSO providers, storage backends), public/ (static assets), and course-specific additions in this fork: ai-logs/, docs/sprint0-4, STUDENTS.md.

How do developers typically contribute? What is the PR and code review workflow? What are the standards for issue reporting, triage, and management?

- Contributing: Open an issue and discuss the approach before writing code; then submit a PR.
- PR/review workflow: Automated checks (linting, type-checking, tests) run on every PR, a reviewer gets assigned, and the PR is merged once approved.
- Issue standards: Bug reports use a structured template and must confirm you searched for duplicates first; feature requests go to Discussions instead; inactive or low-detail issues get flagged and eventually closed.

Screenshots of project running locally:



Screenshots of tests passing:

